Ders İsmi: Veri Yapıları

Ödev No: 2

Hazırlayan;

Adı Soyadı: Ahmet Faruk İkiz

Öğrenci No: b241210040

Dersi Aldığı Grup: 1.Öğretim B Grubu

Dersi Aldığı Öğretim Görevlisi: Dr.Öğr.Üyesi HÜSEYİN DEMİRCİ

İstenenler:

içerisinde sayıların olduğu bir data.txt dosyası olması burdan çekilen satırların 6 gen biçimindeki öncelikli kuyruk yapısındaki 6 yuvaya satır satır BST şeklinde eklenmesi. Kullanıcıdan tur bilgisi istenmesi tek sayılı turlarda normal çıkış, çift sayılı turlarda öncelikli çıkış sağlanması. Postorder sıraya göre çıkarılan bu ağaçlar bir sağındaki kuyruk'un altıgen yapısına tek tek düğümleri dağıtılmalıdır. Ekranda her bir kuyruğun normal çıkış/ öncelikli çıkış değeri gözükmesi istenmektedir. Öncelikli çıkış değeri ağaç yüksekliğine göre olmalıdır

Ödevde Yaptıklarım ve Kurduğum Mimari:

Sabit altıgen yapıyı sağlamak için ve kuyruktaki eleman sayısı hiçbir zaman değişmeyeceği ve görece az sayıda olduğu için ArrayList ile gerçekleştirilmiş dairesel kuyruk yapısı kullandım. Oluşturulan bütün bstler için ayrı heap noktalarında alan açıp pointer yapısı kullanmaktansa yanyana dizili alanlar (dizi) yapısının daha uygun olacağını düşündüm.

Bu kuyrukları tutabilmek için bir pointer dizisi kullandım: "Queue **kuyruklar; kuyruklar = new Queue*[kuyrukSayisi];"

Main.cpp :

İçerisinde Program nesnesi heap alanında oluşturulur ve baslat() metodu çağırılır.

Program.cpp :

Programdaki temel nesneleri oluşturan ve yöneten bir sınıftır. Dosyadan veri okunması; TekTur, ÇiftTur durumları buradan yönetilir. En son EkraniCiz() methodunda kuyruk içindeki değerler hesaplanıp bir depolaması reserve edilmiş bir stringe aktarılır.

Queue.cpp :

Burada array ile gerçekleştirilmiş bir öncelikli kuyruk yapısı bulunur. DequeueMax yapılırken eleman silindikten sonra kaydırma yöntemi kullanılmıştır.

Öncelikle postOrderDequeue ve postOrderDequeueMax fonksiyonları silinecek düğümü bulur bu düğümlerin postordersil methodunu çağırırlar ve bu düğüm her bir elemanını geçici bir diziye aktaracak şekilde postorder bi şekilde kendi içini boşaltır. En son kalan boş düğüm de dequeue edilir ve yerine yeni boş bir ağaç koyulur. Bu sayede altıgen yapı korunmuş olur.

Geçici dizi Program.cpp içerisinden çağırılan verilenAgaciDagit() methoduna parametre olarak gönderilir. Burada şeklin en önünden başlayarak düğümler dağıtılır. Burada ön ve arka kavramı olarak kuyruğun front'u ve kuyruğun back'ini düşündüm. Dairesel bir kuyruk yapısı kullandığımdan ve modüler işlemle front back'i yönettiğimden kısa vadede bazı ağaçlarda fazla birikim olup bazılarında az veri kalmasına neden olabilir.

Bu durum bazı turlarda fazla 1 ve 0 sayılarının gözükmesini açıklıyor. Program ilerledikçe bazı noktalarda çıkacak ağaç, yüksekliği en fazla olan ağaç durumuna geliyor. Bu da aynı ağacın kendine bölünmesiyle 1 sonucunu veriyor.

Bst.cpp :

Recursive fonksiyon mantığıyla bir bst ağacı yazdım. Burada postorder mantığını biraz değiştirip her bir elemanı bulduktan sonra bir dizinin içerisine atacak ve sonrasında bu düğümü silecek şekilde güncelledim.

Öğrendiklerim:

*Postorder dolaşım ve çift pointerlı array yönetimi konusunda deneyimim arttı.

*Dairesel queue yapısını gerçekleştirirken modüler döngüsellik konusunda tecrübe edindim.

Pointer kullanımı ve referans operatörünü kullanmak konusunda deneyimim arttı. Bazı noktalarda "&" şeklinde kullanıp başka bir pointerda adres tutmak yerine direkt o pointerı değiştirmem gereken yerler oldu. Yine postordersil kısmında int versini & operatörüyle yollayıp recursive fonksiyon içerisinde index yönetimi yaptım.

Zorlandığım Kısımlar:

Ödevin mimarisini çözmek ve hangi veri yapısı üzerinden mimari inşa edeceğimi anlamak burada beni en çok zorlayan kısım oldu. Kuyrukları tutmak için bir arraylist yazmayı düşündüm fakat sonra çift pointer bir arrayin yeterli olacağını farkettim.

Kodumun bir noktasında ağacı postorder bir şekilde gezip diziye aktarıp sonra ağacı komple siliyordum fakat bu durumun ödevle birebir uyuşmadığını farkettim. Kodumu postorder gezerken aynı anda düğümü de silecek şekilde güncelledim bunu yaptığımda kodumun işleyişi komple bozuldu sonrasında peekMax() fonksiyonunu iki yerde kullandığımı birincisi kullandıktan sonra o düğümü boşalttığım için ikinci peekMax() ın başka bir düğümü gösterdiğini farkettim. dequeueMax() içindeki peekMax'ı kaldırıp direkt silmek istediğim düğümün pointerını yollayacak şekilde kodu güncellediğimde sorunlar ortadan kalktı.

Eksik Bıraktığım Kısımlar:

Her seferinde height fonksiyonunu çağırmayıp height değerini bir değişken içinde tutsaydım programım daha optimize çalışabilirdi fakat bu kısmın mimarisini tam oturtamadığım için klasik recursive height yöntemini uyguladım.